Technical Documentation

Technical Documentation for *Sheep and Shepherd*. A Bible verse memorization game inspired by *Snake* for the Web. Originally written by Hanwen Luan and Charlie Mikels (the Blob Wizards) for J-Term 2024 Sr. Project.

> ⓘ **Staying up to date**
>
> This file was considered up to date as of January 24th, 2024 (commit `2948cbc`).
>
> If this was a while ago, look for a more up-to-date version in the documentation directory. If this this file is the latest version already, consider reviewing it and if it looks good update the timestamp above.
>
> See About this file.

Table of Contents

Overview

Sheep and Shepherd is an activity built for an in-development Bible memorization program. It was initially developed at Taylor University during J-Term 2024. This Bible memorization program needs a better name, but also will feature a suite of activities and tools to help people memorize Bible verses.

Sheep and Shepherd will be one of those activities, but at time of writing, it's a stand-alone game. In the future it will need to be integrated into the final project, but that's a job for some future developers.

If *you* are one of those future developers: Greetings from January 2024! It was like -5°F about 10 days ago, but it's starting to warm up a little. Anyways, good luck fixing our code! :)

Here are the big files to look out for:

- `index.html` is the core HTML file. Imports `welcome.js` and `styles.css`
- `style.css` css styles, including most UI animations.
- `welcome.js` handles most of the UI, including button event listeners. Imports `game.js`
- `game.js` handles all the Three.js stuff and most of the game logic.

See Files and Directories for a more details.

Development Setup

We use Node and Vite to host and build the app. To get the development environment set up:

1. Install Node on your computer.
2. Clone the repo, and navigate to `Blob Wizards/Sheep Search`.
    - To the future developer: There's a chance that this project has been moved out of it's original repo and into a new one. In this case, navigate to whatever directory has the `index.html`, `game.js`, and `welcome.js` files, and the `public/` directory.
3. run `npm install` to download all of the node dependencies for this project.
4. run `npx vite` to host a development build of the game.
5. This should give you a link to localhost. Open that link in your browser to play the game.
    1. Vite will automatically reload the page when you update a file.

There are a handful of Unicode characters in the source code and branch names. For example: `→`, `↓`, `™`, and `🐑`. Be sure your editor / terminal can correctly render these characters.

To work on the 3D models, you will also want to install Blender. We used blender 4.0, but you should use whatever version is the latest (stable) build. The minimum version is blender 2.8, as that's the earliest version with the necessary GLTF exporter. See public/*.glb.

Also check out Environment Variables.

Extra notes on Vite

- Vite was recommended to us by the Three.js docs. It's not system-critical, but it was

helpful, so we stuck with Vite.

- Vite has some extra optional arguments. Most notably:
  - `--host` will make Vite host the game to the local network, allowing you to test the game on phones and other devices.
  - `--mode MODENAME` will let you change the mode Vite runs it. Notably, this effects what environment variables file gets loaded. See the section on Environment Variables. The default mode is `development`
- Vite is a blunder, and as such, it gets picky about how some files are included.
  - See also: `public/`
  - JS scripts should be marked as `type="module"`.
- use `npx vite build` to bundle the app into a final deliverable. You can preview your most recent build with `npx vite preview`. See the dist/ directory.

> **Broken Build**
>
> At time of writing, the `npx vite build` is broken. (The Sky background isn't included correctly.)

Files and Directories

This section is an semi-high level overview of each of the files in this project. For details on specific variables and functions, refer to the code comments.

blender_assets/*

Collection of `.blend` files used to generate .glb files used by Three.js. Backup blend files (`*.blend1` files) are ignored by .gitignore. Blender specific textures and assets for these files should end up here as well. That said, most of these assets are also used in-game, so they could be stored in the public directory.

See also: GLB files.

dist/*

The output directory of `npx vite build`. It should be a ready-to-deploy build of the full game. When deploying the game to some production server, copy the files from this directory instead of the actual project repo.

You can preview the last build with `npx vite preview`

See also: Extra notes on Vite.

documentation/*

An Obsidian vault containing this file and other documentation related files. At time of writing, it's ~~a little~~ overkill, but `¯\_(ツ)_/¯`. You will find a `.obsidian` directory here. But the `workspace.json` file should be in the gitignore list.

node_modules/*

All the modules downloaded through `npm install`. See also package.json and package-lock.json in this file, and the official About NPM page.

public/*

Vite makes everything in the `public` directory "public" by default. Meaning you can type in `localhost:5173/background1.jpg` and because `background1.jpg` is in the public folder, the user can access it.

Probably. Vite told us to put pictures and things in `public` so we're just following along with their advice.

Despite that these assets are in the `public` folder, Vite actually pretends they're in the root directory when it hosts the file. So if you've got the file `public/favicon.png`, the HTML should say `href="favicon.png"`, without `public`. Vite will let you say `href="public/favicon.png"`, but it will throw a warning in the console about it.

GLB files

Most of the files in `public` should be self explanatory: There's PNGs, MP3s, etc. But GLB may be uncommon.

GLB is the file format used to store 3D models for Three.js. It is the same file format has GLTF, but in an efficient binary format. Blender has a built-in GLB exporter.

> **⚠ Exporting Materials**
>
> Blender throws out a lot of material information when converting to GLB. For best results: Use one principled BSDF material, texture nodes, and one UV map. Getting fancy with mix nodes or multiple UV maps will not translate well by the time it gets to Three.js.
>
> However, the GLB format does support multiple UV maps, and Three.js supports custom shaders. You can get around most of these problems if you did all of the shading in Three.js instead of Blender.

Remember to check your export settings in the right panel in the export file selector.

- Modifiers are not applied by default. Enable them in Data → Mesh → Apply Modifiers.
- Make sure your including the correct objects in the export. By default, everything in the scene is included. The Include section lets you control what gets exported.
- If working with multiple objects with identical mesh data, Enable GPU instances in Data → Scene Graph → GPU instances. This can improve performance in Three.js. but the objects must have linked mesh data.
  - Use `ALT` + `D` instead of `SHIFT` + `D` to duplicate these meshes.

Current GLB files:

- `Sheep.glb`: Includes the sheep character with a rig and a few (currently unused) animations.
- `wolf.glb`: Wolf character. Rigged and has a placeholder animation.
- `Shepherd.glb`: Player's character.
- `Floor.glb`: The big one. Floor includes several objects:
  - The ground and grass tufts are decoration objects and don't do anything.
  - The fence and fence gate mark the playspace boundaries, but are squished to fit the screen on mobile and on narrow windows.
  - A special `PlayspaceRaycast` object. Like the fence, It is also resized to fit the game window's size. But it is also the mouse raycast target and the surface that lost sheep spawn on. It is also invisible to the camera when in game.

package.json and package-lock.json

These files control what packages will be installed by `npm install`. You should not directly edit `package-lock.json`, instead it is built from the `package.json` file.

See NPM's documentation on package.json and package locking.

index.html

It's the HTML file. Sets the initial DOM layout and imports style.css and welcome.js. Notably, it does *not* import game.js.

> **ⓘ Use of classes**
>
> We got in the habit of using classes like IDs really early on. Most elements have a unique class instead of having a unique ID. Similarly, the JS files use `document.querySelector('.classname')` to find elements in the DOM.

Notable elements:

mobile-web-app-capable

```
<meta name="apple-mobile-web-app-capable" content="yes" />
<meta name="mobile-web-app-capable" content="yes" />
```

Tells mobile browsers that when the app is installed to the home screen, it's OK to hide browser chrome. This lets the game run full screen on iOS.

div#whole

```
<div class="whole" id="whole">
```

Contains all elements that show while the game is running. including the game canvases, pause menu, ending confetti, etc. Notably it does not include the welcome screen nor the animated cloud background.

div#game

```
 <div id="game">
        <canvas id="game_canvas"></canvas>
        <div id="game_labels_canvas"></div>
</div>
```

game.js uses 2 renders. One renders the 3D world to the canvas, the other renders the in-game text as `div.game_sheep_label` elements, and puts them inside `div#game_labels_canvas`. This lets us render text as actual text, and use CSS to style the sheep labels.

See also game.js and style.css/.game_sheep_label.

style.css

Most of the css styles. Some styles are manually applied by JS or as in line styles in index.html.

Notable Style Exceptions

- The maroon text on the music and SFX toggles in the pause menu are applied manually by welcome.js.
- Many elements like `.instruction` and `.whole` have their `display` style manually set to `block` or `none` to make them appear and disappear. Be careful when debugging these

display styles.

- the style and animations of the `.displayedSheep` elements (the white word boxes inside the bottom `.displayBoard`) are entirely created inside game.js. See `updateDisplayBoard()` in `game.js`.

Notable Styles

That Blurred Glass Effect

Some elements like `.introduction` and `.displayBoard` have a blurred glass look. This is done by using `backdrop-filter: blur(20px);`. Not all browsers may support this rule. For instance, Webkit browsers have their own rule: `-webkit-backdrop-filter: blur(20px);`

```
.yourElement {
        background: transparent;
        border: 2px solid rgba(239, 239, 239, .5);
        border-radius: 20px;
        backdrop-filter: blur(20px);
        box-shadow: 0 0 30px rgba(0, 0, 0, .5);
        -webkit-backdrop-filter: blur(20px);
}
```

cloud slider styles

`.slider-background {}`, `.slider > img {}` and `@keyframes slide {}` all control the sliding clouds in the background. Each cloud is given an index variable `--i` from `index.html`

The cloud images (`.slider > img`) are crammed into the top left corner, and then we use a bunch of `calc()` equations to offset their height, opacity, and speed. This creates the illusion of depth.

.displayBoard and .displayedSheep

Selects the window at the bottom that shows the list of collected Sheep. Note that game.js manually creates an entire css animation for newly created `.displayedSheep`. See `updateDisplayBoard()`.

`#game`, `#game_canvas`, and `#game_labels_canvas`

`#game` is a holder for the other two. All three fill the entire browser window, and the two canvas elements must be on top of each other for the 2D labels to stay in sync with the 3D sheep.

`#game_canvas` is an actual `canvas` element, however `game_labels_canvas` is just a div that contains the sheep labels. (see `.game_sheep_label` and friends). A window resize event listener manualy sets the resolution and size of the canvas element, so that it always take up the maximum space.

.game_sheep_label and friends

The div elements generated by the CSS2D renderer. They are manually positioned by the renderer with inline `transform` rules.

Related styles:

- `.game_colected_sheep_label` Add-on style for collected sheep. Makes them less bold when looking for the next guessing words.
- `.game_wolf_label` Unused. If player tries to collect an incorrect sheep, this style gets applied instead of `.game_colected_sheep_label`
- `.game_winning_sheep_label` After winning the game, this style is applied to all of the collected sheep, turning the text green and bringing back some visibility.

These sub styles stack. So a winning sheep has the classes `.game_sheep_label`, `.game_colected_sheep_label`, and `.game_winning_sheep_label`.

.confetti

Controls the animations of the individual confetti pieces. Heavy use on `:nth-of-type()` let's us style each confetti "flake" individually.

Other styles

- `.loading`: Greys out and disables the "Play Game" while the game is being initialized.
- Unlike the instruction page, the `.pauseGame` element is not set to `display: none` Instead, it uses `opacity: 0%` and `pointer-events: none;` to simulate the same effect. Apply the `.fade-in` class to unset it and fade in the pause menu.

welcome.js

In charge of most interactions with the UI and most sounds. It also imports game.js into the project and calls a few functions from there.

Audio

`welcome.js` has 3 main sounds it can play:

- Background Music (`bgMusic.mp3`)
- Sheep Sounds (`sheepAudio.mp3` , sometimes called `sheepMusic`)
- Button click sound (`clicking.wav)

Playing these sounds are handled in `playBgMusic()` , `playsheepMusic()` and `playClickSound()` . The first two will try to initialize an audio context with `initAudioContext()` . They will also not play anything if the user isn't looking at the app, or if the user has paused those sounds.

There is a `"visibilitychange"` event listener that re-calls `playBgMusic()` and `playsheepMusic()` . This causes the game to pause the music whenever the player moves to another app, and tries to re-play it whenever they return.

The pause menu has controls to mute the music or sound effects. The user's preference is stored in Local Storage, using the keys `"userPausedMusic"` and `"userPausedSound"` . When the script loads, these preferences are saved as booleans to `userPausedMusic` and `userPausedSound` .

Local Storage only saves `UTF-16` strings as keys. So we're actually storing strings that say `"true"` and `"false"` .

```
let userPausedMusic = false;
// …

if (localStorage.getItem("userPausedMusic") == undefined) {
  localStorage.setItem("userPausedMusic", "false")
} else {
  userPausedMusic = (localStorage.getItem("userPausedMusic") == "true")
  if (userPausedMusic) {
    document.querySelector('.musicControl').style.color = 'maroon';
  }
}
```

Button Listeners

Button on click interactions had to be set up inside of `welcome.js` instead of as function calls inside of the HTML because Vite didn't like it when we did that. >:(

Most of the buttons either toggle the music or sounds, or they call some action to control `game.js`. See Exports to see what `welcome.js` has access to.

Notable patterns:

- Because we're loading data from an API, the UI usually loads faster than the game is ready to play. If you're going to call a game control function, first check if `Game.gameState.gameInitialized` is `true`. If it is not true, you can call `await waitForGameInitialization()` and your function will resume after the game has initialized.
    - Similarly, to stop the game, call it with `await Game.stopGame()` to make sure the game stops all the way before continuing.
    - The initial "play game" button (`.btnInstruction-popup`) unclickable by default
- To reset the game, (as in the `.replay` "replay game" button,) call `await Game.stopGame();` then `Game.playGame();`. See Game Control Functions.
- `playClickSound()` should be added to all the button events. It does it's own `userPausedSound` check, so put it everywhere!

game.js

The core of the game logic. This file is split up into a few sections

- Imports and global vars
- Game control functions and exports
- Initialization
- Event listeners and Input Handling
- Game updates and Animation

The game will automatically initialize itself after the script is loaded.

For notes on how to use a specific function, refer to source code comments.

Imports and APIs

There are 3 important external libraries this script uses:

1. `Three.js` and it's related addons are used to do all of the 3D stuff. Including:
    - Rendering the 3D geometry to `#game_canvas`.
    - Rendering the in-world 2D text to `#game_labels_canvas`.
    - Project 2D mouse position into 3D space.
    - Scatter sheep and things on the surface of the playspace.

2. `Brill` is a lookup table to get the parts of speech for some words. Used to generate better-than-random distraction words.
    - Not all words appear in the lookup table. (especially biblical words like "nebuchadnezzar" and "sandal".) Use `getBrillSafeWord()` for a consistent word-to-key generator. Words that aren't in the table are given the key `PROBLEM-WORD`. These words are usually proper nouns and should be tested with other problem words
3. bolls.life does not appear in the import list, but it's used in `fetchPassageFromAPI()`. It's a free-to-use Bible app and API that we use to get bible verses, and context verses.

> **Moving to the Bible memorization project**
>
> The larger Bible memorization project will be in charge of feeding our app with verse data. bolls.lifeand `fetchPassageFromAPI()` acts like a placeholder backend until the final product is ready to be a backend. When the real backend is ready, `fetchPassageFromAPI()` should be redirected away from bolls.life and to the new server.

Global Variables

Most of these vars are shorthand vars because I don't want to dig through the scene graph every time I want to reference the raycast object. Similarly, the sheep object is spawned on the fly and isn't part of the scene, so we need to hold onto it separately anyways.

The most important parts of this section are the `gameState` and `player` objects.

- `gameState` is created by `initGameState()` and keeps track of the verse, collected sheep, and score counters. You can find the player's progress through the verse by comparing the length of `gameState.collectedSheep` to `gameState.verse`. It's also exported for use in `welcome.js`.
- The `player` object keeps a few important player vars organized together, most of which are related to the shepherd's movement. Note that player's position and rotation are *references* to the shepherd model's position and rotation. You can update one, and it'll update the other's

Game Control Functions

`game.js` will start initializing itself as soon as the script loads. However it will start is a paused state. Use these control functions to start, stop, or pause the game. All of these functions are exported and can be called from other scripts.

- `playGame()` starts the game for the first time. It spawns the event listeners, starts the

update and animate loops, and spawns the first wave of sheep.
    - `playGame()` will try to full screen the app if it thinks it's running on a mobile device.
    - `playGame()` checks if the game is initialized. If it's not, it will wait for the game to finish before continuing.
    - `playGame()` is async. If you have code that must be ran *after* the game starts, make sure to await this function
- `resumeGame()` is like `playGame()`, but it skips the initialization check, and does not spawn a wave of sheep. Use this to un-pause the game.
- `stopGame()` stops the game entirely. It resets the world back to their initialized states.
- `pauseGame()` stops the control event listeners and the animate loop, but it does not reset the game.
    - Pause game is async. remember to call it with `await` if you need to run code after it.
- to restart the game, call `await stopGame();` then `playGame();`.
- `waitForGameInitialization()` is a helper function that returns a promise. That promise is done once the game is initialized, so you can `await` this function and when it returns, you'll know the game is initialized.

Mobile Browsers

There are a handful of undesirable behaviors that mobile browsers have. Notably, the browsers will slide their URL bars out of the way when you drag over them. To fix this in game, our code will attempt to full screen the window on mobile devices. This hides all of the browser chrome, and thus gets rid of any undesirable movement.

In case the user exits fullscreen, there are some CSS rules to limit the scrolling behaviors anyways. Mostly the `overflow: hidden;` rule in the `body` styles does a lot of heavy lifting. But it (probably?) won't stop "pull-down-to-reload" or other gestures.

> ⓘ **iOS Safari**
>
> Among other challenges, Safari on iOS does not let us go fullscreen. However, you can install the web app to the homescreen, where iOS will see the mobile-web-app-capable tag in the HTML, and then iOS hide the browser chrome when launched from the home page.

🐞 `exitFullscreen()`

Currently, the way to do a quick game reset involves calling `await stopGame();` then `playGame()`. However, `stopGame()` assumes you're totally done with the game, and will try to exit full screen. Since `playGame()` immediately re-enters fullscreen, this creates an unpleasant flash.

To fix the flash, our game will never un-fullscreen itself. (Yes. `exitFullscreen()` is a lie.) Ideally, we'd have a dedicated reset function, or stopGame could have a parameter that controls fullscreen. But for now, the user will have to manualy un fullscreen on these platforms.

(This only really effects Android, since the fullscreen functions are ignored on desktop, and iOS Safari doesn't do fullscreen anyways.)

Game Initialization

`gameInit()` is the big initialization function. There are other helper functions, but those are called through this one. The basic steps are:

- Set up the Three.js scene, camera, and both renderers
- Load the GLB files and set up the 3D environment.
  - A lot of this set up is split between the loader callback functions, and the `environment_setup()` functions.
  - The shepherd and sheep take up about one square meter of space. The wolf is a bit larger. The max-size playspace is about 21.6m by 18.8m, but can shrink to fit the window width.
- Resize playspace for mobile
- Initialize the `gameState{}` object.
  - Sets up lost / collected sheep lists.
  - Gets passage data
- Initializes `player{}`
- Update the `gameInitialized` bool, and Remove the "loading" class from the "Play Game" button (`.btnInstruction-popup`)

`game.js` manually calls `gameInit()` at the end of the script.

Multiple renders

There's multiple ways to render text in Three.js. We followed the advice on the three.js documentation and since we needed the text to stay on top of the sheep objects, we're using a `CSS2DRenderer`. To render the in-world text as actual DOM elements.

Passage Data

We're using an external API called Bolls.life to get Bible verse information. In the future, this will need to be replaced with whatever api we need to use when *Sheep and Shepherd* is moved to the full Bible verse memorization project thing. In the mean time: this is how it works:

1. `passageInit()` calls `fetchPassageFromAPI()`, which will return a `passageObject{}`
    1. `fetchPassageFromAPI()` checks for Environment Variables to control verse selection.
    2. If reference is a string, use regex to parse out the book name, starting chapter, starting verse, etc.
    3. **Validate.** Make sure selected range looks correct.
    4. Ask API for a list of books and translations. Get the Book ID for our chosen book.
    5. For every chapter in our selected text, ask the API for the entire chapter.
    6. Duplicate the return. Truncate one into the test passage string. The other gets turned into into the context string.
    7. return `passageObject`
2. Generate part-of-speach lookup tables from passage.
    - One table is word to POS, the other is POS to valid words. Both get stored in the passage object
    - Many words aren't in the Brill lookup table. Use `getBrillSafeWord()` to trim unusual words into words that might be in the Brill table.
        - `getBrillSafeWord()` returns an object that gives the "key" word, and the associated parts of speech.
        - Some words might make it through this function and still might not exist in `Brill`. These words are given a `"PROBLEM_CASE"` part of speech, so that they match with other problem words.

Event listeners and Input Handling

These functions are called by event listeners.

- `resizeCanvas` is called whenever the window is resized. It keeps the camera and renderer resolutions in sync to fill the browser window.
- `updatePointerPos` is called when the mouse moves. It recalculates the Mouse's in-world position by calling `pointerToFloorPos()`
  - `updatePointerPos()` also updates the player's `lookTarget`.
  - If the user is touching the screen, or is holding down Mouse 1, then also update the move target.
- A simple click event will not trip the mouse move event. `UpdatePointerAndMove()` will update the mouse position, and then immediately move there.
  - These is special logic to see if the user actually clicked on a sheep label. In these cases, the shepherd should walk directly to the sheep. (The label usually appears slightly above the sheep. So clicking on the word would send the shepherd *behind* the sheep, instead of collecting it.)
- The control input listeners are not enabled by default. Use `startInputListeners()` and `pauseInputListeners()`. to control them. (not exported. Outside of `game.js` you should use the Game Control Functions instead.)

Update / Animate loop

All of the gameplay logic is handled in the `updateGame()` function.

1. Get the delta from `updateClock`.
   - This is the only time update clock should be checked in the entire script. `updateClock` belongs to `updateGame()` (except when stopped or started with the game control functions)
2. Update some archaic debug objects.
3. Attempt to move the player.
4. If the player was not moved:
    1. See if we're near a lost sheep. (Nearly on top of one)
        1. If the nearby sheep is the correct sheep, it is collected with `collectSheep`, and a new wave is spawned
            1. If the nearby correct sheep is the last sheep to guess, don't spawn a new wave, and instead show the end-game popup.
        2. If the nearby sheep is incorrect, remove one already collected sheep (if any), and spawn the wolf. After it's done just standing there menacingly, remove it and spawn a new wave.

5. Go down the list of collected sheep, starting with the one closest to the shepherd. If that sheep is too far from it's follow target, step towards it.

The `updateGame()` function does not loop itself. Instead, it is called by `animate()`, which does loop itself recursively. Animate can be stopped with the Game Control Functions, or by setting `gameState.runState` to anything but `"running"`.

> ⚠ **Be careful not to start the animate loop more than once at a time**

Sheep and Wolves

Several helper functions are used to spawn sheep and wolves

`getValidSpawnPoints()` asks the play space for locations where it can spawn a sheep. It will try again and again if it's too close to the player or another lost sheep.

`spawnSheep()` creates a new sheep at the given position with the given text.

`deleteSheep()` destroys a sheep. How could you! >:(

`spawnWolf()` and `deleteWolf()` are similar. However, the `spawnWolf()` can take an initial Y rotation, and `deleteWolf()` has no paramiters, since there will only be one wolf on screen at a time.

> ⚡ **`spawnWolf()` is a liar**
>
> `spawnSheep()` actually creates a new copy of the sheep model and places it in the world. But since we're bundling the wolf's animation data inside of the wolf object, we can't clone the wolf object. So `spawnWolf()` doesn't actually spawn the wolf, but rather it teleports the wolf to the new location. Similarly, `deleteWolf()` will teleport the wolf somewhere underground.
>
> This is partly why, at time of writing, the Sheep aren't animated.

`clearLostSheep()` destroys all the lost sheep on the screen, including incorrect and un-collected correct sheep. Remember to collect any sheep before clearing them.

`colectSheep()` adds a given sheep to the collected sheep list, and applies relevant styles. **It does not remove the sheep from the lost sheep list.** Remember to call this function like this:

```
sheep = gameState.lostSheep[i];
gameState.lostSheep.splice(i,1);
collectSheep(sheep);
```

`spawnNewSheepWave()` clears all the lost sheep, and spawns 3 new sheep. One sheep will have the correct next word on it, the other two will have distraction words based on the part of speech look up tables in `gameState.passage`. The distraction words are guarantied to be unique, but if there aren't enough distraction words, the distraction sheep will fail to spawn. See also Passage Data.

UI Control

A handful of UI related things are updated in `game.js`.

- `updateDisplayBoard()` updates the bottom `.displayBoard` element with the list of words the player has collected. This function actually resets the entire board each time it's called, and adds an animation to the last element.
- `updateUi()` updates the counter and timer under the logo in the top-left corner.
- `updateCounter()` updates the number in the sheep counter by the `delta` amount. You'll usually call it after collecting or loosing a sheep with `1` or `-1` respectively. You can pass it `0` to update the score display without changing the number.
- `updateScore()` is similar to `updateCounter`, but it keeps track of your total number of guesses. Score is not shown to the user, except on the end screen after collecting the last word in the verse.
- `resetScoreAndCounter()` set both back to `0`
- `showEndGamePopup()` and `awayEndGamePopup()` are similar to the Button Listeners in welcome.js. They show and hide the `#finishGame` element. the `showEndGamePopup()` function also starts the confetti.
- `makeItConfetti()` makes it confetti. 🎉🎉🎉

Environment Variables

Node and Vite let us launch our app with environment variables to make development easier.

Current Vars

- `VITE_SKIPWELCOME`: Boolean. Game will click the "Play Game" button (`.btnInstruction-popup`) for you as soon as the game is done initializing
- `VITE_CUSTOMREFERENCE`: String. Forces `fetchPassageFromAPI()` to always use this verse instead of whatever it was going to do.

- `VITE_DEBUGPASSAGE` Boolean. Forces `fetchPassageFromAPI()` to use the built-in debug string. This is the same effect as passing `"debug"` to `fetchPassageFromAPI()`, or by setting `VITE_CUSTOMREFERENCE` to `"debug"`.
- `VITE_DEBUGOBJECTS`: Boolean. Enables a few debug objects that should probably be removed.
    - The green cube represents the mouse-to-world raycast intersection.
    - The red arrow is the direction to the Shepherd's look target.

Using Environment variables

In the code, you can look for environment variables with `import.meta.env.VITE_YOURENVVAR`, where `VITE_YOURENVVAR` is your vite environment variable

To conveniently set environment variables, create a file called `.env.development` and put your variables in there like this:

```
VITE_SKIPWELCOME: true
VITE_CUSTOMREFERENCE: "John 11:35" # "Jesus wept"
```

Vite in development mode (the default mode) will automatically load these variables when it serves your app. You can also manually set Vite's mode with `--mode`, and vite will load whatever `.env` ends with the name of the mode.

For example, `npx vite --mode charliedev` will load variables from `.env.charliedev`, and not `.env.development`.

Make sure to gitignore your development `.env` files.

About this file

This file is likely to be either a Markdown file, or a PDF file.

The Markdown version is part of an Obsidian vault, and is written in Obsidian flavored markdown. You should be able to read this file with any markdown editor / renderer. But I strongly recommend opening the documentation folder as a Obsidian vault and reading this file from there.

The PDF version is created from the above markdown version of this file. To export a new PDF, open the updated markdown version in Obsidian, then run the `Export to PDF` command. (Open the Command Pallet with `CTRL` + `P` and type `PDF`. It should appear in the list.) The default export options should be fine.

This Obsidian vault (currently) comes with one community plugin: Automatic Table of Contents. It's included because Obsidian's default PDF exporter is kinda under-powered, and doesn't correctly generate internal links, nor header information for PDF viewers to generate their own outline. This plugin lets us include an automatically-updating Table of Contents. It won't render correctly in regular markdown renderers. But a regular markdown renderer would have all of the heading data back, so it should be able to generate it's own outline.

⚠ Themes

The PDF export will inherit any themes and CSS snippets that are currently applied to Obsidian. Some themes don't expect to be printed and kinda look bad. For ideal results, set obsidian to the default light theme before exporting.

At time of writing, there are a few included and tested css snippets that **should be enabled** to make the exported PDF a bit nicer nicer.

Commercial use

Obsidian is free for personal use, and the devs are pretty chill. But if *Sheep and Shepherd* somehow becomes a commercial product, we will need to either pay for a commercial license or stop using Obsidian.

To get rid of Obsidian stuff, delete the `.obsidian` directory inside of the documentation directory. You'll then be left with plain markdown files.

Licensing info can be found here.

---

kthxbai :)